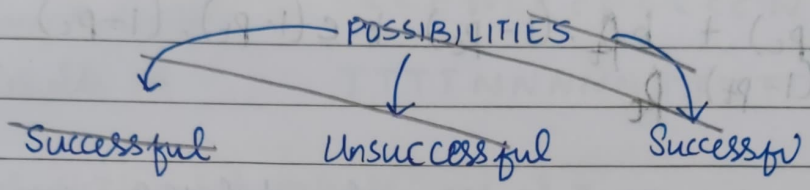
4/3/24

BRANCH PREDICTION

* Prediction works well for unconditional branches
* Prediction creates problems for conditional branches
  ∵ In a conditional branch, the loops are executed thousands of times before they are exited, prediction may or may not be useful



	Branch ~~Fab~~	Predicted	
Successful	Taken	Taken	p(success)
Unsuccessful	Not taken	Taken	$= \frac{1}{2}$
Successful	Not taken	Not taken	
Unsuccessful	Not taken	Not taken	= p(failure)

↗ cost + additional cycles

* Cost or penalty is associated with branches
  ↳ total number of cycles required

Penalty = Cost − 1

Result

Branch cost (number of cycles required)

$$C_{ave} = a\,(p_{\text{branch predicted taken and taken}}) + b\,(p_{\text{branch predicted not taken but taken}}) + c\,(p_{\text{branch predicted but not taken}}) + d\,(p_{\text{branch predicted not taken and not taken}})$$

Cost of an instruction $= (1 - p_b) + p_b\, C_{ave}$

$$C_{ave} = a(p_t \cdot p_c) + b\,p_t \cdot (1 - p_c) + c(1 - p_t) \cdot (1 - p_c) + d\,(1 - p_t) \cdot p_c$$

Assuming when there are N cycles when there is a successful outcome ($a = d = N$) and B cycles when there is an unsuccessful outcome ($c = b = B$)

$$C_{ave} = (1 - p_b) + p_b\,[N \cdot p_c + B(1 - p_c)]$$

Static Branch Prediction

- assumes that branch is always taken or not taken at compilation
- achieves upto 75% accuracy
- Static branch can be extended using branch prediction bit in opcode which can be set or reset by compiler

* h-hint can also be appended. An accuracy between 74-94%

Dynamic branch prediction

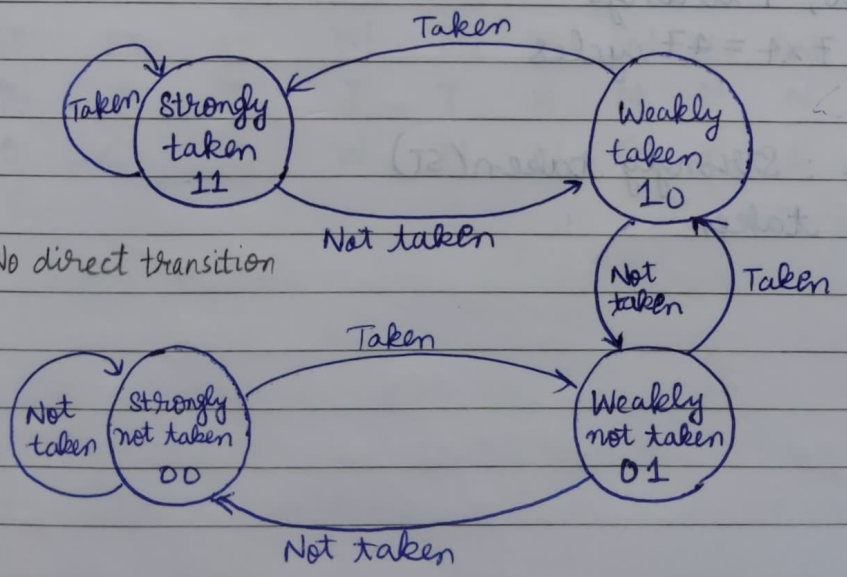
- operate at runtime using the past behaviour of the program to predict its future behaviour
- table can be maintained for each branch
- 1-bit / 2-bit prediction
- 1-bit works better for loops and can be worst for occasional branches

Initial assumptions T T T N N N N T T T T T N N N
compared with C C C W C C C W C C C C C W C C
Results T T T T N N N N N T T T T T N N

prediction bit initially set as T
flipped to N when assumption does not match result

2-bit branch predictor

- uses recent history of a branch to predict its future behaviour
- required per branch / separate for each branch



1-bit

Initial assumption T

Sequence T T T N T T T T N N N N N T N N N T T T T N T T T T T → 16
C C C W W C C C W C C C C W W C C W C C C C W W C C C N → 10

⇒ 8 wrongs, 18 correct

Branch taken 4 cycles, not taken 1 cycle

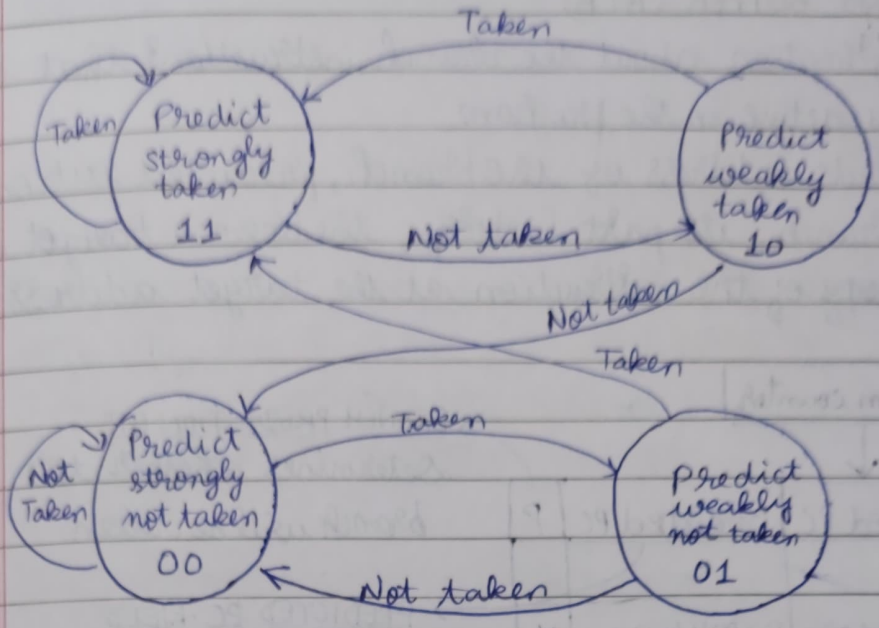
Without prediction, required number of cycles = 16 × 4 + 10 × 1
= 74 cycles

With prediction, required number of cycles = 18 × 1 + 8 × 4
= 50 cycles

2-bit

| | | |

2-bit predictor with hysterisis (just for comparison)



Actual	T	T	T	N	N	T	N	T	N	N

Saturating counter branch predictor

State	ST	ST	ST	WT	WN	WT	WN	WT	WN	SN
Prediction	T	T	T	T	T	N	T	N	T	N
Outcome	C	C	C	W	W	W	W	W	W	C

Hysteresis

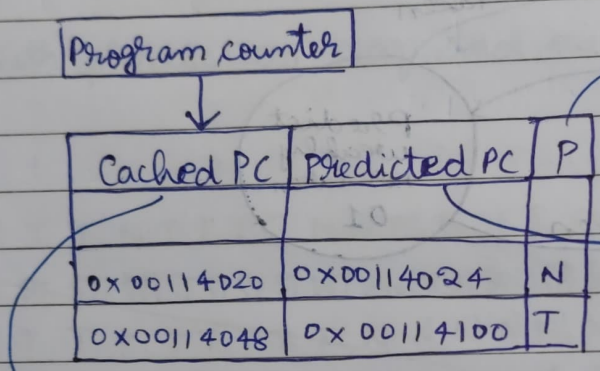
State	ST	ST	ST	WT	SN	WN	SN	WN	SN	SN
Prediction	T	T	T	T	T	N	N	N	N	N
Outcome	C	C	C	W	W	W	C	W	C	C

5/3/24

BRANCH TARGET BUFFER (BTB)

- stores information about the branch instructions that are currently active in the program
- can include address of the branch, predicted outcome of the branch, its past history, the branch target address and a copy of the instruction at the target address

Simple Form of BTB



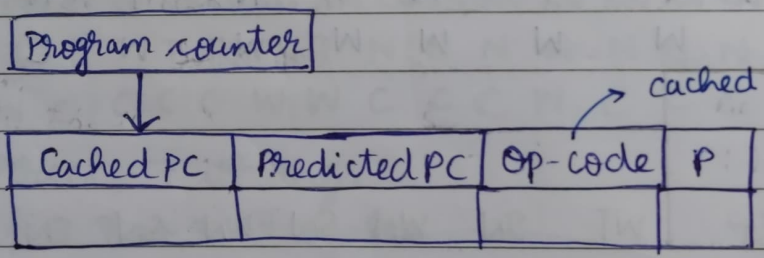
BRANCH PREDICTION BIT
Determines whether this branch will be taken

PREDICTED PC FIELD
Provides address of the next instruction

CACHED PC FIELD
Contains the most recently accessed branch addresses

* Misprediction costs 2 cycles

Modified BTB

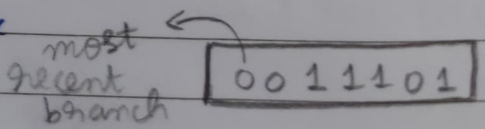


cached opcode of mispredicted branch address

* Misprediction costs 1 cycle

2-level Branch Prediction

- The prediction is quite challenging when branches interact with each other
- There might be a degree of correlation b/w branches
- BRANCH PATTERN HISTORY BPH is used as a means of observing the interaction between branches (12-bit pattern usually)
- Written into shift register



LHT - keeps track of all the outcomes of branches which are mapped to a single entry

Make use of Branch History to select a column based on global sequence of branches and branch address to get the index

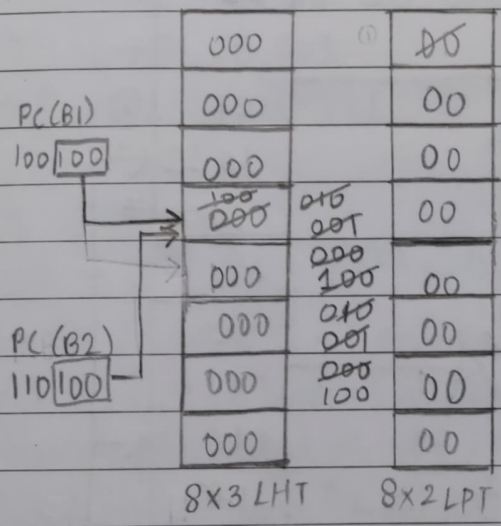
6/3/24

CORRELATING BRANCH PREDICTION

- Prediction accuracy may be improved if the recent behaviour of other branches is taken into consideration
- (m, n) predictor uses the behaviour of the last m branches to choose from 2^m branches predictors

LHT Local History Table
LPT Local Prediction Table

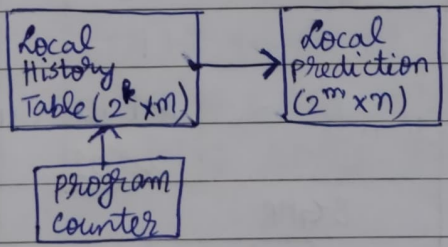
★ Shift LHT right by 1 and prepend → 0 if outcome==N, 1 if outcome==T (in LHT)



★ MSB always indicates latest outcome of the branch that was executed

Prediction N N N N N N N T
Outcome T N N N T N N N T

In LHT 100 ⇒ 010 (0)
In LHT 000 ⇒ 100 (1)



BRANCH HISTORY
B1 : TNTNTNTNTN...
B2 : NNNNNNNNN...
↳ never taken

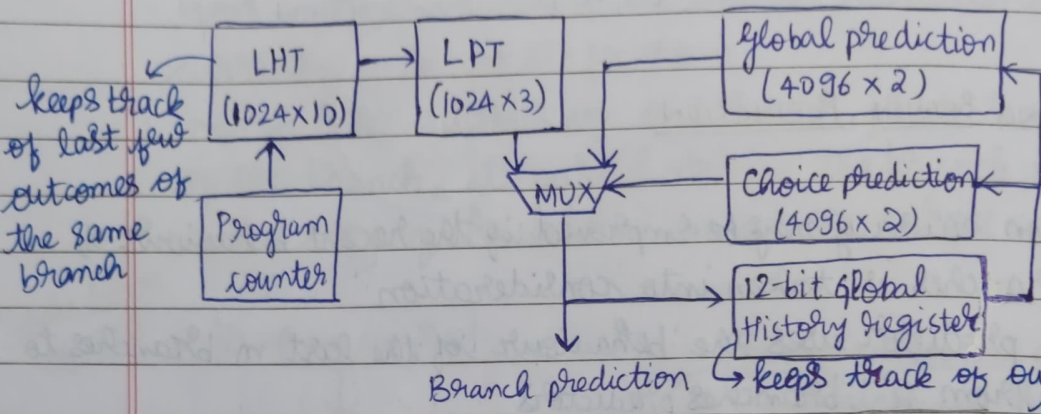
CODE

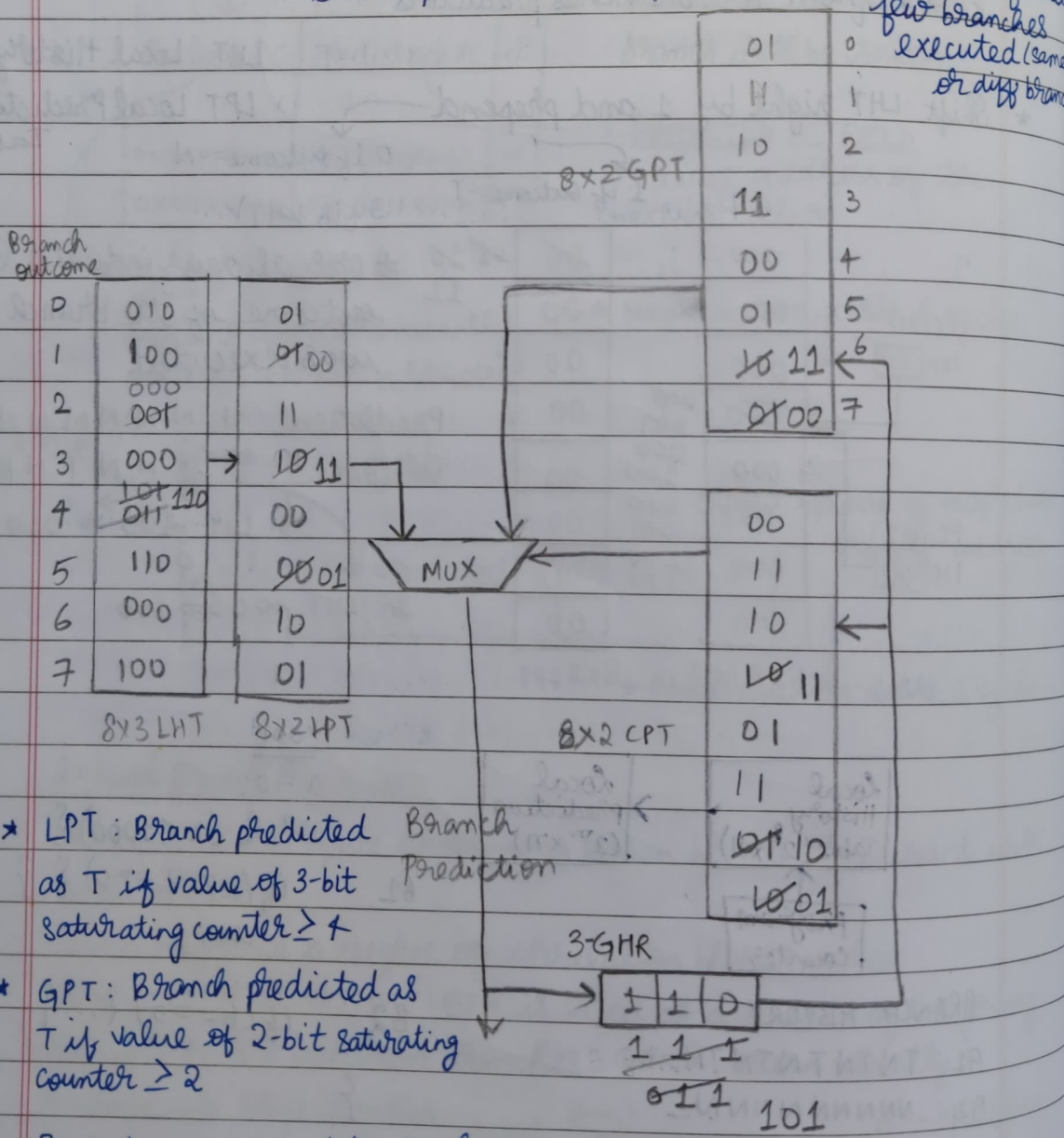
```
a = 0;
while (a < 1000) {
B1    if (a % 2 == 0) { . }
      a++;
      ...
B2    if (b == 0) { ... }
}
```

GHT - keeps track of all the outcomes of branches executed so far

TOURNAMENT BRANCH PREDICTOR

Adaptively combines local & global branch predictor behaviour





* LPT : Branch predicted as T if value of 3-bit saturating counter ≥ 4
* GPT : Branch predicted as T if value of 2-bit saturating counter ≥ 2
* Prediction made if both local & global predictions are same
* Use Choose Prediction Table on mismatch → Global if ≥ 2 saturating counter value
  → Local if < 2 saturating counter value

PC (B2) 100 010

LHT @ 010 ⇒ 001

LPT @ 001 ⇒ 01 ⇒ Not Taken

CPT

Branch taken ⇒ increment

Branch not taken ⇒ decrement

Mismatch → 00, 01 → local pred; 10, 11 → global pred

I PC 100 100

- Index into LHT @ 5th entry (4th index)
  Value = 011 ⇒ Goto 3rd location in LPT (4th entry)
- Value in LPT = 10 ⇒ Local prediction is T
- Check GHR value 110 ⇒ 6th index in GPT (7th entry)
  Value in GPT = 10 ⇒ Global prediction is T
- Local and global predictions match ⇒ Branch predicted as TAKEN
- Update corresponding LPT value to 11 (increment by 1); same in GPT
- Increment CPT to 10 (inclined towards global)
- LHT 101|1 ⇒ 101
- GHR 111|0 ⇒ 111

II PC 100 010

- LHT ⇒ 2nd index (3rd entry) Value = 001
- Go to 1st index (2nd entry) in LPT Value = 01 ⇒ local pred = N
- GHR 111 ⇒ 7th index (8th entry) in GPT Value = 01 ⇒ Global pred = N
  Branch predicted as NOT TAKEN
- Updation LPT ⇒ 01 to 00 (decremented)
  GPT ⇒ 01 to 00 (decremented)
  CPT ⇒ 10 to 01 (decremented) {inclined towards local}
  GHR ⇒ 011|1 ⇒ 011
  LHT ⇒ 000|1 ⇒ 000

III PC 110 100 LHT 101 → LPT 00 ⇒ local pred N

GHR 011 → GPT 11 ⇒ Global pred T

Conflict ⇒ Go to CPT 10 ⇒ inclined towards global prediction

Branch predicted as TAKEN GPT unchanged CPT++ ⇒ 11

LPT ++ 01 GHR 101|1 ⇒ 101 LHT 110|1 ⇒ 110

Inst. producing result	Inst. using result	Latency in C.C
FP ALU OP	FP ALU OP	3
FP ALU OP	store double	2
load double	FP ALU OP	1
load double	store double	0

11/3/24

No new instructions are fetched or issued until the dependencies are cleared.
→ Use stalls
→ pipeline scheduling

DYNAMIC SCHEDULING

Hardware rearranges the instruction execution to reduce the stalls while maintaining data flow and exception behaviour.

→ it can tolerate unpredictable delays
→ but increase in hardware complexity
→ Basic idea: have multiple functional units
→ inorder fetching

DECODE STAGE: talks about dependencies
→ Leads to out-of-order completion; out-of-order execution
→ out-of-order introduces possibility of WAR and WAW hazards

Status register: Flag, mode

- An exception is imprecise if the processor state when the exception is raised does not look exactly as if the instructions were executed in sequential order

1D pipeline stage

Issue
→ Decode instructions, check for structural hazard

Read operands
→ wait until no data hazards & then read operands

- If dependencies are present, it goes into queue of pending instructions (FIFO order)
- Issue is in order, can bypass in 2nd stage → out-of-order execution

TOMASULO'S ALGORITHM

- handles anti and output dependencies by renaming the registers dynamically : Avoid WAW & WAR
- handles speculation, i.e., reduces control dependencies and takes corrective action when prediction is wrong.
- algorithm is proceeded as if there are multiple functional units.

```
DIV.D   F0, F2, F4          DIV.D   F0, F2, F4
ADD.D   F6, F0, F8          ADD.D   S, F0, F8
S.D     F6, 0(R1)     →     S.D     S, 0(R1)
SUB.D   F8, F10, F14        SUB.D   T, F10, F14
MUL.D   F6, F10, F8         MUL.D   F6, F10, T
```

Q.

```
for (i=999; i>=0; i=i-1)
      x[i] = x[i] + s;
```

```
Loop:  fld     f0,0(x1)
       fadd.d  f4,f0,f2
       fsd     f4,0(x1)
       addi    x1,x1,-8
       bne     x1,x2,Loop
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Without Scheduling – Introducing Stalls matching with latency

```
Loop:  fld     f0,0(x1)      1
       stall                 2
       fadd.d  f4,f0,f2      3
       stall                 4
       stall                 5
       fsd     f4,0(x1)      6
       addi    x1,x1,-8      7
       bne     x1,x2,Loop    8
```

(Clock cycle issued)

Total cycles = 8 x1000 =8000
Number of stall cycles = 3 /8
Total stall Cycles = 3 x1000 =3000

Loop Overhead = 2/8
Total loop overhead Cycles = 2 x1000= 2000

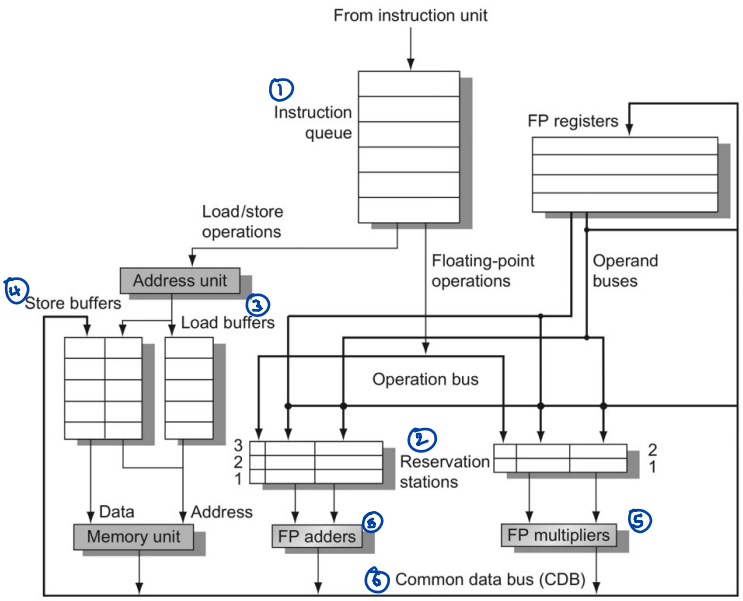
With pipeline Scheduling

```
Loop:  fld     f0,0(x1)
       addi    x1,x1,-8
       fadd.d  f4,f0,f2
       stall
       stall
       fsd     f4,8(x1)
       bne     x1,x2,Loop
```

out-of order execution

Total cycles = 7 x1000 =7000
Number of stall cycles = 2 /7
Total stall Cycles = 2 x1000 =2000

Loop Overhead = 2/7
Total loop overhead Cycles = 2 x1000= 2000



① Insts. are sent from instr. unit into instr. queue from which they are issued in FIFO order

② It includes operations & actual operands as well as info used for detecting & resolving hazards

③ They hold comp. of effective address until computed
Traces outstanding loads waiting on memory
Holds results of computed loads waiting for CDB

④ They hold components of EA until its computed
Holds destination memory address of outstanding stores
Holds address & value to store until memory unit available

⑤ The FP adders implement + & -
The FP multipliers implement x & ÷

⑥ All results from FP units/Load units are put on CDB
They further go to FP Register File, reservation stations & store buffers

→ Steps in Tomasulo Execution

1) Issue
   → Instruction is taken from instruction queue
   → It checks if reservation station is free
      ↳ If yes, instruction placed in station
         Register names replaced with reservation station tags
         This performs register renaming
      ↳ If no, Issue stalls

2) Execute
   → Instruction waits until all operands are ready
   → This happens because reservation station monitors CDB
   → When operations arrive, instruction begins execution & Multiple executions may execute simultaneously

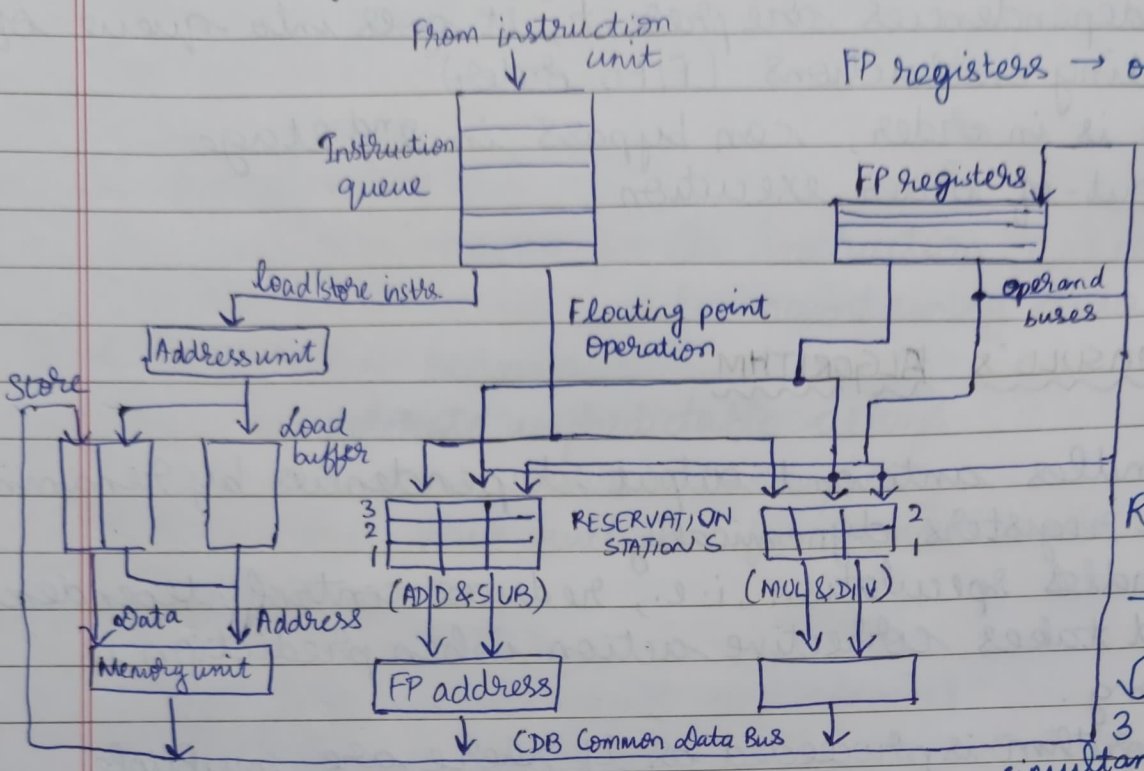
3) Write Result
   → When execution finishes, Result sent to CDB
   → Then registers are updated & reservation stations that were waiting receive it.

→ Handling Loads & Stores
   → Loads/Stores require 2 stages
   i) Address Calculation - EA = Base + Offset
      - Stored in Load/Store buffer
   ii) Memory Operation - Load executes when memory unit is free
      - Store waits until address & value are ready

→ Benefits of Tomasulo
   i) Eliminates RAW Hazards
   ii) Eliminates WAR Hazards
   iii) Eliminates WAW Hazards
   iv) Enables high ILP

→ Limitation: Dynamic Scheduling introduces 1 extra cycle latency because result must be written to CDB & then picked by next instruction

MIPS Floating Point using Tomasulo's Algorithm



Effective address : Base address + offset

RAW { W: ADD R4, R0, R8 ; X: MUL R2, R0, R4 } WAW & RAW

RAW { Y: ADD R4, R4, R8 ; Z: MUL R8, R4, R2 }

ASSUMPTIONS

- FLOS to dispatch up to 2 instructions in every cycle
- Instructions can begin execution in same cycle that is dispatched to a RS
- ADD → 2 cycles & MUL → 3 cycles

CYCLE 1

gives provider's address

RS (ADDER) supplier

		Tag1	S1	Tag2	S2
W:	1.	0	6.0	0	7.8
	2.				
	3.				

RS (MUL/DIV)

		Tag1	S1	Tag2	S2
X:	4.	0	6.0	1	–
	5.				

Floating point register (FLR)

	Busy	Tag	Data
0			6.0
2	Yes	4	3.5
4	Yes	1	10.0
8			7.8

W & X dispatched in order

CYCLE 2

		Tag1	S1	Tag2	S2
W:	1.	0	6.0	0	7.8
Y:	2.	1	–	0	7.8
	3.				
X:	4.	0	6.0	1	–
Z:	5.	2	–	4	–

	Busy	Tag	Data
0			6.0
2	Yes	4	3.5
4	Yes	12	10.0
8	Yes	5	7.8

Y & Z dispatched in order

CYCLE 3

1.
2. 0 13.8 0 7.8
3.
4. 0 6.0 0 13.8
5. 2 — 4 —

0			6.0
2	Yes	4	3.5
4	Yes	2	10.0
8	Yes	5	7.8

CYCLE 4

1.
2. 0 13.8 0 7.8
3.
4. 0 6.0 0 13.8
5. 2 — 4 —

0			6.0
2	Yes	4	3.5
4	Yes	2	10.0
8	Yes	5	7.8

CYCLE 5

1.
2.
3.
4. 0 6.0 0 13.8
5. 0 21.6 4 —

0			6.0
2	Yes	4	3.5
4	~~Yes~~	~~2~~	~~10.0~~ 21.6
8	Yes	5	7.8

CYCLE 6

1.
2.
3.
4.
5. 0 21.6 0 82.8

0			6.0
2	~~Yes~~	~~4~~	~~3.5~~ 82.8
4	Yes	5	21.6
8			7.8

CYCLE 7

1.
2.
3.
4.
5. 0 21.6 0 82.8

0			6.0
2			82.8
4			21.6
8	Yes	5	7.8

CYCLE 8

1.
2.
3.
4.
5. 0 21.6 0 82.8

0			6.0
2			82.8
4			21.6
8	Yes	5	7 8

CYCLE 9

1.
2.
3.
4.
5.

0			6.0
2			82.8
4			21.6
8			

18/3/24

RESERVATION STATION

Each reservation station has 7 fields:

* Op ⇒ operation to perform on source operands S1 & S2
* Q_j, Q_k ⇒ The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already in V_j or V_k or is unnecessary
* V_j, V_k ⇒ The value of source operands
  Only one of the V fields or Q field is valid for each operand
  For loads, V_k field is used to hold the offset field
* A ⇒ used to hold information for the memory address calculation for a load or store
  Holds immediate field of instruction initially
  holds effective address after address calculation
  (accumulator)
* Busy ⇒ indicates that this reservation station & its accompanying functional unit are occupied

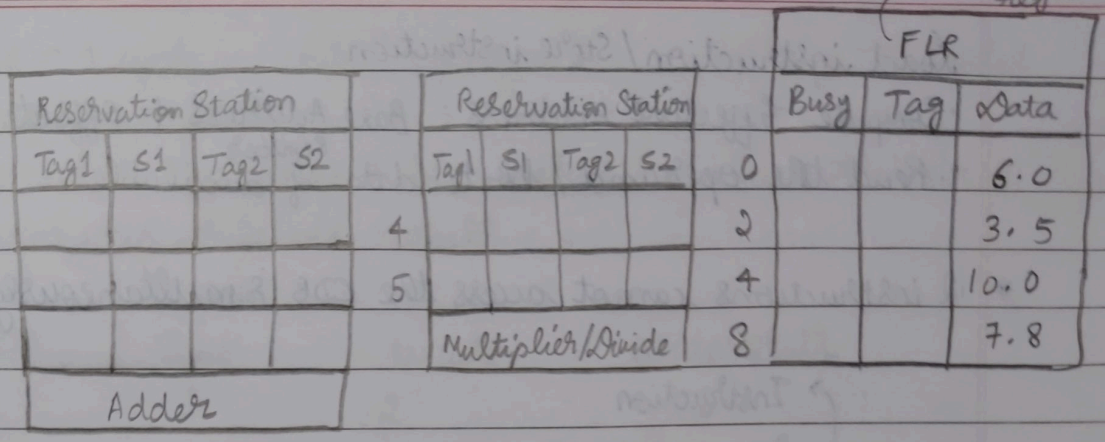
```
w   ADD R4, R0, R8 ]  RAW (w & x)
x   MUL R2, R0, R4 ]  WAW (w & y)
y   ADD R4, R4, R8 ]  WAR (y & z)
z   MUL R8, R4, R2 ]
```

WAW → resolved by renaming

WAR → resolved by renaming

* WAW & WAR can be resolved by renaming the registers
* RAW cannot be resolved by register renaming

ASSUMPTIONS
- FLOS to dispatch up to 2 instructions in every cycle
- Instructions can begin execution in time



* Q_i ⇒ The number of the reservation station that contains the operation whose result should be stored in this register

Resource Availability for Tomasulo's Architecture

Assume 2 integer units, 3 FP add units, 2 FP multiply units

1) L.D F6, 32(R2)
2) L.D F2, 44(R3)
3) MUL.D F0, F2, F4
4) SUB.D F8, F2, F6
5) DIV.D F10, F0, F6
6) ADD.D F6, F8, F2

* ID (Instruction decode) → issue (waits when there is structural hazard)
* ID (Instruction decode) → Read operands (provided data dependence is solved)

* Write result is associated with Common Data Bus (CDB)

→ Show what the info tables look like for the given code sequence when only the first load has completed & written its result

2 & 3 ⇒ RAW (true data dependency)
2 & 4 ⇒ RAW
3 & 5 ⇒ RAW
4 & 6 ⇒ RAW
5 & 6 ⇒ WAR
.....

Regs[ ] Refer to the register file
Mem[ ] Refer to memory

Load instruction / Store instruction
- Compute Effective Address = Base Register Address + offset
- Read the operand / Write data if available

* 2 instructions cannot access the CDB simultaneously

Status → Instruction
Status → Reservation
Status → Register

Q. Assume the latencies involved with the execution of
Load → 2 cycles
MUL → 6 cycles
ADD/SUB → 3 cycles
DIV → 10 cycles
Explain in detail how many cycles the sequence of code given with single issue using MIPS architecture & Tomasulo's algorithm. Assume without overlap if nothing has been specified for load

	Issue	Execute	Write result
Load	1	2-3	4
Load	2	4-5	6
MUL	3	6-11	12
SUB	4	6-8	9
DIV	5	12-21	22
ADD	6	9-11	13

(both cannot write onto CDB simultaneously)

19/3/24

Load → 1 cycle ; Add → 2 cycles Multiply → 6 cycles
Divide → 12 cycles

Show what the status tables look like when the MUL.D is ready to write its result

	Issue	Execute	Write result
LD1	1	2	3
LD2	2	3	4
MUL	3	4 - 9	10
SUB	4	5 - 6	7
DIV	5	stall 10 - 21	22
ADD	6	7 - 8	9

can get the result in same cycle

```
Loop:  L.D     F0, 0(R1)
       MUL.D   F4, F0, F2
       S.D     F4, 0(R1)
       DADDIU  R1, R1, -8
       BNE     R1, R2, Loop
```

	Issue	Execute	Write result
LD	1	2	3
MUL	2	3 - 8	9
SD	3	stall 9	10
DADDIU	4	stall 10 - 11	12
BNE	5	stall 12 - 14	15

* A load and a store can be safely done out of order, provided they access different addresses.

Both load & store access the same address
- → Load before store ⇒ WAR hazard
- → store before load ⇒ RAW hazard

* Interchanging 2 stores to same address results in WAW hazard

HARDWARE SPECULATION

Drawbacks of Tomasulo's algorithm

- cannot go back once result has been written
- requirement of large amount of hardware
- each reservation station must contain an associative buffer, which must run at high speed, as well as complex control logic
- Single CDB

20/3/24

Exploiting More ILP

- Branch prediction reduces stalls but may not be sufficient to generate the desired amount of ILP
- One way to overcome control dependencies ⇒ SPECULATION

Hardware Based Speculation

Extends the idea

Speculating with Tomasulo

⇒ Separate execution from completion
⇒ Instruction Commit
⇒ Allow instructions to execute & complete out of order but force them to commit in order
⇒ Add a hardware buffer (REORDER BUFFER ROB) with registers to hold the result of an ~~intermediate~~ instruction between completion & commit {FIFO queue in order issued}

★ ROB

- Sits b/w Execution & Register File
- Source of operands
- integrated with Store buffer
- Reservation Stations use ROB slots as a tag
- Instructions commit at head of ROB FIFO queue

ROB Data Structure

* INSTRUCTION-TYPE FIELD
  ↳ Indicates whether the instruction is a branch, store or register operation
* DESTINATION FIELD
  ↳ Register number for loads, ALU ops, or memory address for stores
* VALUE FIELD
  ↳ Holds the value of the instruction result until instruction commits
* READY FIELD
  ↳ Indicates if instruction has completed execution and the value is ready

Instruction Execution

1. ISSUE Get an instruction from Instruction Queue
2. EXECUTION Operate on operands (EX)
3. WRITE RESULT Finish execution (WB)
4. COMMIT Update register or memory with the ROB result

Example Assume a load operation takes 2 cycles, ADD/SUB take 2 cycles, MUL takes 6 cycles and DIV takes 12 cycles. Assume a single-issue processor where a load operation can start when the memory unit is available and no overlap with the write cycle and read cycle on CDB.

Solve using ROB.

do w & w/o speculation 2 issue

	Issue	Execute	Write result	Commit
L.D F6, 32(R2)	1	2-3	4	5
L.D F2, 44(R3)	2	4-5	6 (write)	7 (being written in diff way)
MUL.D F0, F2, F4	3	7-12 (read)	13	14
SUB.D F8, F6, F2	4	7-8	9	15
DIV.D F10, F0, F6	5	14-25	26	27
ADD.D F6, F8, F12	6	10-11	12	28

multiple functional units

not 6 :: no overlap b/w write & read

⇓ in-order commit

Using only Tomasulo's

	Issue	Execute	Write result
L1	1	2-3	4
L2	2	4-5	6
MUL	3	6-11	12
SUB	4	6-7	8
DIV	5		
ADD	6		

Issue

Cycle 1 LD F6, 32(R2) ✓

Reservation status

	Busy	Op	V_j	V_K	Q_j	Q_K	A
	Yes	Load	–	–	–	–	32 (holds only imm)

Register status

	F6
	Load1

	Issue	Execute
~~LD 1~~		
Cycle 2 LD F2, 44(R3)	✓	✓

L1	Yes	Load	–	–	–	–	32+Regs[R2]
L2	Yes	Load	–	–	–	–	44

F2	F6
~~Load2~~	~~Load1~~

	Issue	Execute	Write result
Cycle 3 LD 1	✓	✓	
LD 2	✓		
MUL	✓		

L1	Yes	Load				32 + Regs[R2]
L2	Yes	Load				44
MUL	Yes	MUL	–	Regs[F4]	Load2	–

F0	F2	F6
Mult	Load2	Load1

	Issue	Execute	Write
Cycle 4 LD1	✓	✓	✓
LD2	✓	✓	
MUL	✓		
SUB	✓		

L1	Yes	Load1	–	Regs[32+Regs[R2]]	–	–	–	
L2	Yes	Load2	–	Regs[F4]	Load 2	–	44+Regs[R3]	
MUL	Yes	MUL	–			–	–	
SUB	Yes	SUB	–		Load2	Load1	–	

F8
Add1
(SUB)

		Issue	~~Execute~~	Write res
Cycle 5	L1	✓	✓	✓
	L2	✓	✓	
	MUL	✓		
	SUB	✓		
	DIV	✓		

L1	No	Load1
L2	Yes	~~Load2~~
MUL	Yes	~~MUL~~
SUB	Yes	~~SUB~~
DIV	Yes	DIV

25/3/24

MULTIPLE ISSUE PROCESSOR

Flavors
- Statically scheduled superscalar processor
- VLIW (very long instruction word) processor
- Dynamically scheduled superscalar processors (out-of-order execution)

COMMON NAME	ISSUE STRUCTURE	HAZARD DETECTION	SCHEDULING	DISTINGUISHING CHARACTERISTIC
Superscalar (static)	Dynamic (variable number of issues)	Hardware	Static	In-order execution
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, no speculation
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation
VLIW/LIW	Static	Primarily software	Static (All hazards determined & indicated by compiler (often implicitly))	
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined & indicated explicitly

Basic VLIW Approach

- use multiple, independent functional units
- packages multiple operations into one very long instruction or requires that the instructions in the issue packet satisfy the same constraints

5 operations contained in the instructions
⇒ 1 integer operation
⇒ 2 floating-point operations
⇒ 2 memory references

* Functional unit 16-24 bits per unit
* Instruction length 80-120 bits

Exercise $x[i] = x[i] + S$

INSTRUCTION PRODUCING RESULT	INSTRUCTION USING RESULT	LATENCY
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

i^{th} element itself is being updated, there is no dependency across the iterations ($(i+1)^{th}$ or $(i-1)^{th}$ elements not involved)
→ Loop can be unrolled

MIPS

```
Loop : L.D     F0 , 0(R1)        * 9 clock cycles
       stall
       ADD.D   F4 , F0, F2
       stall   stall
       S.D     F4 , 0(R1)
       DADDUI  R1 , R1, #-8
       stall
       BNE     R1, R2, Loop
```

Unrolled

```
L.D     F0 , 0(R1)
L.D     F6 , -8(R1)        * Unroll factor of 4
L.D     F10 , -16(R1)
L.D     F14, -32(R1)
ADD.D   F4, F0, F2
ADD.D   F8 , F6, F2
ADD.D   F12 , F10, F2
ADD.D   F16 , F14, F2
S.D     F4 , 0(R1)
S.D     F8 , 8(R1)
DADDUI  R1 , R1, #-32
S.D     F12 , 16(R1)
S.D     F16 , 8(R1)
BNE     R1, R2 , LOOP
```

VLIW

Mem ref 1	Mem ref 2	FP operation 1	FP operation 2	Integer op/ branch
L.D F0, 0(R1)	L.D F6, -8(R1)			
L.D F10, -16(R1)	L.D F14, -24(R1)			
L.D F18, -32(R1)	L.D F22, -40(R1)	ADD.D F4, F0, F2	ADD.D F8, F6, F2	
L.D F26, -48(R1)		ADD.D F12, F10, F2	ADD.D F16, F14, F2	
		ADD.D F20, F18, F2	ADD.D F24, F22, F2	
S.D F4, 0(R1)	S.D F8, -8(R1)	ADD.D F28, F26, F2		
S.D F12, -16(R1)	S.D F16, -24(R1)			DADDUI R1, R1, #-56
S.D F20, 24(R1)	S.D F24, 16(R1)			
S.D F28, 8(R1)				BNE R1, R2, LOOP

- 9 cycles
- 2.5 ops per cycle ($\frac{23 \text{ ops}}{9 \text{ cycles}}$)

56-32
56-24
56-48

* VLIW has an overhead in the form of increased number of FP registers

$CPI = \frac{1}{3}\left(\frac{9}{7}\right) = 0.428$

↓

∴ effectively there are 3 instructions (L.D, S.D, ADD.D)

26/3/24

Exploiting ILP Using Dynamic Scheduling, Multiple Issue and Speculation

Extend Tomasulo's algorithm to support multiple issues

Exercise

```
Loop:  LD      R2, 0(R1)
       DADDIU  R2, R2, #1
       SD      R2, 0(R1)
       DADDIU  R1, R1, #8
       BNE     R2, R3, Loop
```

2-issue processor
- once without speculation
- once with speculation

Assume there are separate integer functional units for effective address calculation, for ALU operations & for branch condition evaluation. Assume that up to 2 instructions of any type can commit per clock. Create a table with first 3 iterations of this loop.
↳ 2 CDBs

Load → 2 cycles Integer op & branch → 1 cycle Store → 1 cycle

Iterations are not overlapped

No overlap between read and write

classmate
Date
Page 90

without speculation

Out-of order write

ITERATION NO.	INSTRUCTIONS	ISSUE	EXECUTE	MEMORY ACCESS	WRITE CDB	COMMENT
1	LD R2, 0(R1)	1	2	3	4	First issue
	DADDIU R2, R2, #1	1	5		6	Wait for LD
	SD R2, 0(R1)	2	3	7		Wait for DADDIU
	DADDIU R1, R1, #8	2	3		4	Execute directly
	BNE R2, R3, Loop	3	(7)			Wait for DADDIU
			↗ wait till BNE of iteration 1 executes			
2	LD R2, 0(R1)	4	8	9	10	Wait for BNE
	DADDIU R2, R2, #1	4	11		12	
	SD R2, 0(R1)	5	9	13		
	DADDIU R1, R1, #8	5	8		9	
	BNE R2, R3, Loop	6	13			
3	LD R2, 0(R1)	7	14	15	16	
	DADDIU R2, R2, #1	7	17		18	
	SD R2, 0(R1)	8	15	19		
	DADDIU R1, R1, #8	8	14		15	
	BNE R2, R3, Loop	9	19			

with speculation

ITERATION NO.	INSTRUCTIONS	ISSUE	EXECUTE	READ ACCESS	WRITE CDB	COMMITS	COMMENT
1	LD R2, 0(R1)	1	2 (EA of R1)	3 (value @ 0(R1))	4 (written into R2)	5	1st issue
	DADDIU R2,R2,#1	1	5 (Reads updated R2)		6 → 7 (write updated R2 value)	7	Wait for LD
	SD R2, 0(R1)	2	3 (EA of R1)			7	Wait for DADDIU
	DADDIU R1,R1,#8	2	3		4	8	Commit in order
	BNE R2,R3, Loop	3	7			8	Wait for DADDIU
			updated R1 value @ end of 4				
2	LD R2,0(R1)	4	5	6	7	9	No execute delay
	DADDIU R2,R2,#1	4	8		9	10	Wait for LD
	SD R2, 0(R1)	5	6			10	Wait for DADDIU
	DADDIU R1,R1,#8	5	6		7	11	Commit in order
	BNE R2,R3, Loop	6	10			11	Wait for DADDIU
3	LD R2,0(R1)	7	8	9	10	12	Earliest possible
	DADDIU R2,R2,#1	7	11		12	13	Wait for LD
	SD R2, 0(R1)	8	9			13	Wait for DADDIU
	DADDIU R1,R1,#8	8	9		10	14	Executes earlier
	BNE R2,R3,Loop	9	13			14	Wait for DADDIU

Not waiting on outcome of branch in order to proceed with the iterations, finishes execution faster (fewer number of cycles)

* Main advantage of Superscalar processors is that most of things are maintained using the Reservation Stations & ROB, and not many general-purposes registers are required

- THREAD → Smallest possible job that can be recognisable by the OS
  ↳ will allocate the resources
- Threads belonging to a process share a data segment
- Every process has its own data segment
- Easier to switch b/w threads than it is b/w processes

27/3/24

BRANCH TARGET BUFFERS (BTB)

A branch-prediction cache that stores the predicted address for the next ~~st~~ instruction after a branch is taken

RETURN ADDRESS PREDICTORS

- Caches the most recent return addresses

⇒ pushing a return address on the stack at a call & popping one off at a return

- Sufficiently large cache → will predict the returns perfectly

INTEGRATED INSTRUCTION FETCH UNITS

- a separate autonomous unit that feeds instructions to the rest of the pipeline
- integrates several functions

(1) Integrated branch predictor
(2) Instruction prefetch
(3) Instruction memory access & buffering

SPECULATIONS

Issues

(1) Speculation Support : Register renaming vs. Reorder buffer
(2) How much to speculate

Speculation is not free; costs time and energy
Recovery of incorrect speculation further reduces performance

Speculating through Multiple Branches
↳ a very high branch frequency
↳ significant clustering of branches
↳ long delays in functional units

(3) Speculation & the ~~challege~~ Challenge of Energy Efficiency

- instructions that were speculated & whose results were not needed → energy wasted
- restoring state of processor

(4) Value prediction

THREAD

THREAD → Smallest sequence of programmed instructions that can be managed by a scheduler which is part of OS

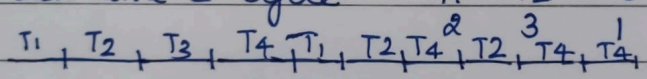
MULTITHREADING → allows multiple threads to share the functional units of a single processor in an overlapping fashion and helps in exploiting TLP

Hardware approaches to multithreading

★ FINE-GRAINED MULTITHREADING

- Switches b/w threads on each clock, causing the execution of instructions from multiple threads to be interleaved
  ↳ often done using ROUND ROBIN

Slack time 1 cycle



PROCESS/JOB	ARRIVES AT	COMPLETES EXECUTION
T1	1	5
T2	2	8
T3	3	3
T4	4	10

DISADVANTAGE → slows down the execution of an individual thread

★ COARSE-GRAINED MULTITHREADING

- Capable of executing a single thread at a time; able to fully switch to execute a different thread without software intervention
- limited in its ability to overcome throughput losses.

★ SIMULTANEOUS MULTITHREADING (SMT)

- a variation of fine-grained multithreading
- implemented on top of a multiple-issue, dynamically scheduled processor
- uses TLP to hide long-latency events in a processor (↑ses usage of functional units)

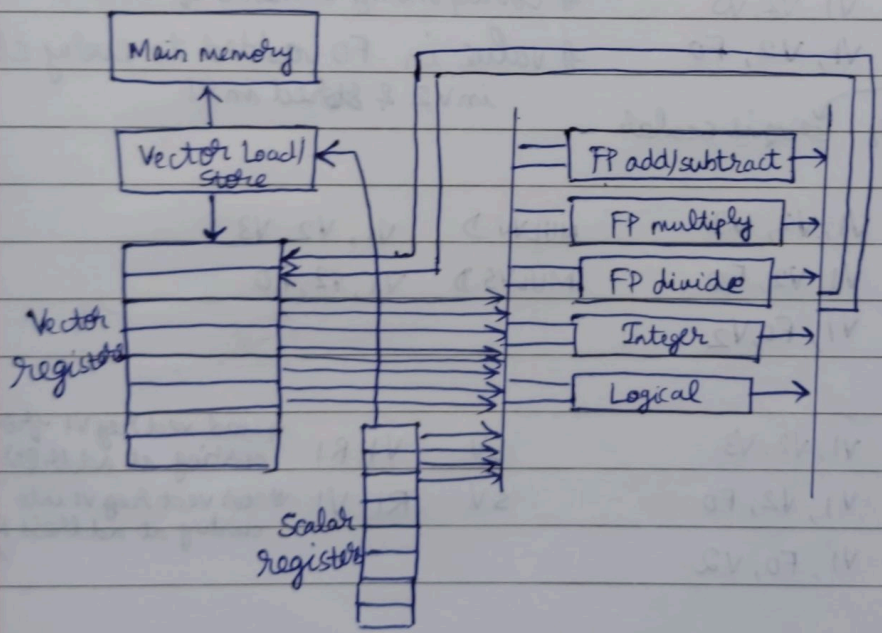
8/3/24

DATA-LEVEL PARALLELISM (DLP)

- Requirement of SIMD has come from matrix oriented computations, applications like image & sound processing
- SIMD are more power efficient than MIMD
- Types for study on DLP are
  - Vector architectures
  - SIMD with multimedia extension (MMX, SSE and AVX extensions)
  - GPU (Heterogeneous architecture)

Vector architecture

- take set of data elements from memory, processing in large register files and put them back into memory
- Vector loads and stores are deeply pipelined, reduces overall memory latency.



VECTOR REGISTERS

- fixed length bank holding single vector
- VMIPS has 8 vector registers, can hold 64 elements each 64 bits
- Need multiple ports for read/write

VECTOR FUNCTIONAL UNITS

- fully pipelined
- a control unit for hazard detection

VECTOR LOAD/STORE UNITS

- loads & store vector from/to memory
- can move with a bandwidth of 1 word per clock
- can handle scalar load & store

SET OF SCALAR REGISTERS

- can provide input to vector functional unit
- 32 general purpose registers & 32 floating point registers

2 additional registers used
↳ Vector length register
↳ Vector mask register

```
ADDVV.D   V1, V2, V3     # corresponding elements of V2 & V3 added
ADDVS.D   V1, V2, F0     # value in F0 added to every element
                           in V2 & stored in V1
```

~~dest~~ is vector ~~src2~~ is scalar
src1

```
SUBVV.D   V1, V2, V3      MULVV.D   V1, V2, V3
SUBVS.D   V1, V2, F0      MULVS.D   V1, V2, F0
SUBSV.D   V1, F0, V2

DIVVV.D   V1, V2, V3      LV   V1, R1   # Load vect reg V1 from memory
DIVVS.D   V1, V2, F0                      starting at address R1
DIVSV.D   V1, F0, V2      SV   R1, V1   # Stor vect reg V1 into memory
                                          starting at address R1
```

MERITS
- in order execution
- lesser design complexity
- operates on vector data
- high performance at low power
- flexible vector size

Example

$Y = a * X + Y$ (a: scalar, X: vector)

MIPS Code

```
        L.D     F0, a          ; 64 x 8
        DADDIU  R4, Rx #512
Loop:   L.D     F2, 0(Rx)
        MUL.D   F2, F2, F0
        L.D     F4, 0(Ry)
        ADD.D   F4, F4, F2
        ~~S.D   F4, F4, F2~~
        S.D     F4, 0(Ry)
        DADDIU  Rx, Rx, #8
        DADDIU  Ry, Ry, #8
        DSUBU   R20, R4, Rx
        BNEZ    R20, Loop
```

VMIPS Code (reduces instr. bandwidth)

```
        L.D     F0, a
        LV      V1, Rx
        MULVS.D V2, V1, F0
        LV      V3, Ry
        ADDVV.D V4, V2, V3
        SV      V4, Ry
```

1/4/24

Vector Execution Time depends on
- Length of operand vectors
- Structural hazards among operations
- Data dependencies

INITIATION RATE : Rate at which vector unit consumes new operands

CONVOY : Set of vector instructions which can be executed together

* Execution time of single vector unit is the vector length

CHIME → Unit of measurement ; unit of time taken to execute 1 convoy

On Execution
- Instructions in a convoy must not contain structural hazards
- If there is hazard in convoy, it must be serialized & put in different convoys

- Convoys of instructions should complete before other instructions
- RAW hazard has to be in different convoys
- CHAINING allows a vector operation to start as soon as individual vector operand element is available
  → allows read & write from same vector register but for different element
- FLEXIBLE CHAINING: chaining to any vector instruction

Convoys = m
Vector length = n
} ⇒ m * n clock cycles

* m convoys take m chimes to execute

Assume copy of each vector functional unit
How many chimes will the VMIPS code for Y = a * X + Y take?
How many FLOP cycles per FLOP are needed?

1. LV V1, Rx
   MULVS.D V2, V1, F0
   } grouped into single convoy
   RAW is not an issue due to chaining
   No structural hazards since separate functional units

2. LV V3, Ry
   ADDVV.D V4, V2, V3
   } grouped into next convoy
   - Chaining
   - No structural hazards
   - LV V3, Ry is not a part of the first convoy ∵ there is only 1 structural unit for Load/store operations
   - V2 available ∵ convoy 2 begins execution after convoy 1 is done

3. SV V4, Ry
   ↳ convoy 3

* Overhead could be 2-3 clock cycles, which is negligible

3 convoys ⇒ 3 chimes
2 FLOP (ADD & MUL) ⇒ 3/2 = 1.5 chimes per FLOP
vector length = 64 (standard) ⇒ 64 × 3 = 192 clock cycles

MULTIPLE LANES

- Concept of parallel pipelines
- N elements to take part of arithmetic instructions
- Reduces number of clocks for execution
- Each lane can do computations independently

- Requires additional hardware

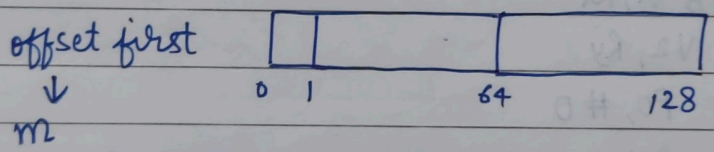
VECTOR LENGTH REGISTERS

- To support data of variable length (VMIPS → supports data length other than 64bits)
- Usually length of vector registers considered as max vector length (MVL)

vector length > MVL ⇒ STRIP MINING

$\frac{129}{64} = 2$ with an offset of 1 $\quad 129 \,\%\, 64 = 1$

Handle the offset first



0 m m+MVL m+2×MVL . . . (n-MVL)

```
low = 0;
VL = (n %. MVL)
for (j = 0; j <= (n/MVL); j = j+1)
    for (i = low; i < (low + VL); i = i+1)
        Y[i] = a * X[i] + Y[i];
    low = low + VL;
    VL = MVL;
```

VECTOR MASK REGISTERS (VMR)

- any case IF statement, vectorization of code is not easy

```
for (i=0; i<64; i++)
    if (X[i]!=0)
        X[i] = X[i] - Y[i];
```

Inner loop cannot be executed unless IF statement is checked

* Conditional execution is possible under vector mask control where Boolean vector is used

When VMR is enabled, vector execution done only on elements whose corresponding entries are in VMR
↳ holds the value 0-1 on each entry

Appends 1 if condition satisfied
0 if condition not satisfied

↓
mask bit

Entries in VMR with zero are unaffected by vector operation

```
LV      V1, Rx
LV      V2, Ry
L.D     F0, #0
SNEVS.D V1, F0
SUBVV.D V1, V1, V2
```

2/4/24

MEMORY BANKS

- load and store happens from/to memory at the rate of one access per cycle
- Memory needs to supply data at this rate which is done by dividing memory into banks
- Memory banks can support data access that are not sequential access
- Can help in dividing memory based on multiple processors

Example The largest configuration of a Cray T90 (Cray T932) has 32 processors, each capable of generating 4 loads and 2 stores per clock cycle. The processor clock cycle is 2.167 ns, while the cycle time of the SRAMs used in the memory system is 15 ns. Calculate the minimum number of memory banks required to allow all processors to run at full memory bandwidth.

32 processors → 6 references per processor {4 load + 2 store}
⇒ Max number of memory references each cycle = 32 × 6
= 192

Each SRAM bank is busy for 15/2.167 = 6.92 ≃ 7 processor clock cycles

∴ A min. of 192 × 7 = 1344 memory banks is required

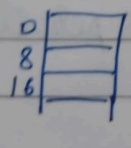
Cray T932 actually has 1024 memory banks
If the cycle time of SRAMs used in memory system is halved, i.e., made 7.5 ns ⇒ $\frac{7.5}{2.167}$ = 3.461 ≃ 4 processor clock cycles

⇒ 192 × 4 = 768 memory banks required, which is serviceable by the Cray T932.

STRIDES

- holds the offset

For floating-point, 8 bytes per element



UNIT STRIDE → can be used for auto-incrementation
Stride factor = 8

- Distance separating elements of single vector is called STRIDE
- Vector architecture can handle non-unit strides effectively
- Vector strides can be stored in general purpose registers

LVWS ⇒ Load Vector With Stride
SVWS ⇒ Store Vector With Stride

* Worst possible stride is a value that is a multiple of number of memory banks
∴ Every access to memory will collide with prev. access

classmate
Date
Page 102

- Strides > 1 complicate memory system

When multiple access contend for a bank, memory bank conflict occurs

$$\frac{\text{Number of banks}}{\text{GCD(Stride, Number of banks)}} \leq \text{Bank busy time}$$

Example 8 memory banks
Bank busy time of 6 clocks
A total memory latency of 12 cycles
How long will it take to complete a 64-element vector load with a) stride = 1 b) stride = 32

a) stride = 1 $\quad \frac{8}{GCD(1,8)} = \frac{8}{1} = 8$

$8 \not< 6 \Rightarrow$ Memory conflict will not occur

initial overhead ← $12 + 64 = 76$ clock cycles

$\frac{76}{64} = 1.18$ clock cycles per element

b) stride = 32 $\quad \frac{8}{GCD(32,8)} = \frac{8}{8} = 1 < 6$ ✓

→ stall on remaining elements ⇒ Memory conflict

$12 + 1 + 63 * 6 = 13 + 378 = 391$ clock cycles

↓ no stall on first access

$\frac{391}{64} = 6.109375$ clock cycles per element

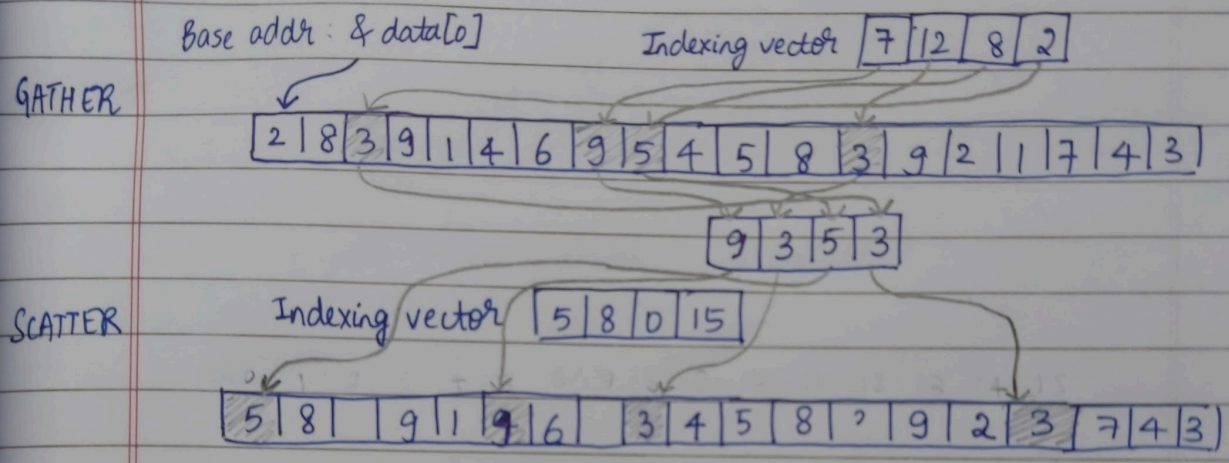
GATHER SCATTER : Handling sparse matrices

Sparse matrices A & C

```
for (i=0; i<n; i=i+1)
    A[K[i]] = A[K[i]] + C[M[i]];
```

where K, M } index vectors representing non-zero elements of A & C

- done using indexing method



LVI (load vector indexed)
- Gather operation takes an index vector & fetches the vector whose elements are based on given base addr + offsets given by index vector ⇒ COMPRESSED FORM

SVI (store vector indexed)
- After operation ⇒ stored in EXPANDED FORM

PROGRAMMER AID Tool used with compiler to enhance efficiency of program

SIMD Extensions

- fix number of data operands in the opcode
- does not have features like strided access & gather-scatter access (non-continuous memory access)
- does not have vector mask register feature

AVX → 256-bit

4D ⇒ Four double precision operands

UNIT 4

3/4/24

GRAPHICS PROCESSING UNITS (GPUs)

Many small cores, Parallel execution, Simple control logic & More ALU, Best for data parallel tasks

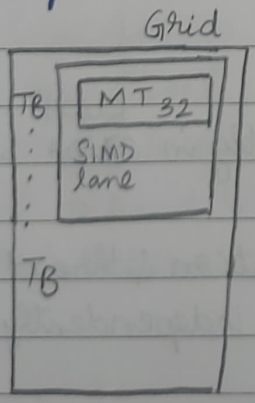
- are known for high performance computing at low cost
- do not use concepts of CPU much
- supposed to be graphics accelerators

COMPUTE UNIFIED DEVICE ARCHITECTURE (CUDA)

- the environment for programming GPUs
- C-like language featuring support for heterogeneous computing and multi-faceted parallelism
- CUDA thread is lowest level programming primitive
- CUDA programming model ⇒ SINGLE INSTRUCTION MULTIPLE THREAD (SIMT)
- THREAD BLOCK ⇒ Block of 32 CUDA threads
- Hardware that executes Thread Block ⇒ MULTITHREADED SIMD PROCESSOR

* Functions for GPU : __device__ or __global__
Functions for system processor : __host__

* CUDA variables declared as in __device__ functions are allocated to the GPU memory, which is accessible by all multithreaded SIMD processors



The extended function call syntax for the function name that runs on the GPU is

name <<< dimGrid, dimBlock >>>
(... parameter list ...)

```
int main(){
fun1 <<1, 10>>();}
```

```
__global__ void fun1{
int tx = threadIdx.x;
printf("%d", tx);}
```

C code ↘ (It has loop where each iteration becomes independant thread)

(C)
```
void daxpy(int n, double a, double *x, double *y)
{
    for(int i=0; i<n; i++)
        y[i] = a*x[i] + y[i];
}
```

```
daxpy(n, 2.0, x, y)    // Invoke the function.
```

(CUDA)
```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n + 255)/256;
daxpy<<<nblocks, 256>>>(n, 2.0, x, y);

// DAXPY in CUDA
__device__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i<n) y[i] = a*x[i] + y[i];
}
```

- 256 CUDA Threads per Thread Block
- Synchronization is not a problem

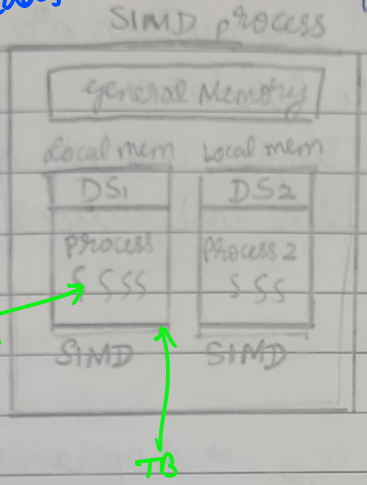
- Programmers specify parallelism explicitly in ~~CUDA~~ CUDA by grid dimensions & number of threads (Block Id, Block Dim, Thread Id)
- GPU hardware handles parallel execution & thread management
- Thread block to be able to execute independently & in any order
- Preferred to have group of 32 threads for better control flow (warp)

Thread - Smallest unit of execution. One instr. at a time
↳ executed by complete CPU unit

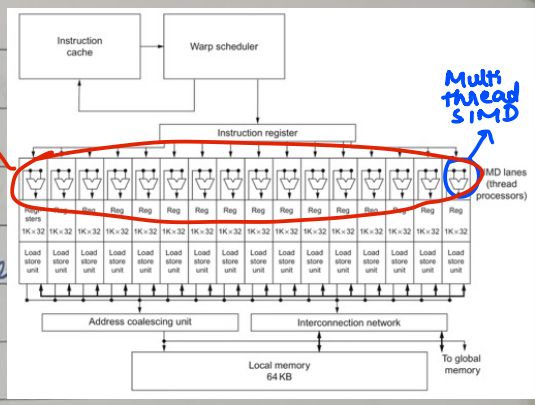
grid → A vectorizable loop made up of one or more TB that can execute in parallel

Thread Block → A vectorized loop executed on a multithreaded SIMD processor, made up of one or more threads of SIMD instruction

↙ Group of 16 SIMD Threads



Warp : Set of threads that execute same instr. of diff. data elements
Mapped to Functional Units



grid is the code consisting of set
Multiplication → Grid has 8192 ele
One Thread Block has 512 elements
⇒ $\frac{8192}{512}$ = 16 thread blocks

4/4/24

GPU Computation Structures

- At low level, it is thread of SIMD instructions with its own PC running on multithreaded SIMD processor
- SIMD thread scheduler uses scoreboard to schedule & assign threads to processor

- Thread scheduler picks a thread for SIMD processor since they are independent to each other
- It keeps track of these by using scoreboard, tracks upto 48 threads & has own PC

GPU Hardware → Thread block Scheduler
- Assigns TBs to multithread SIMD Processors
→ SIMD Thread scheduler
- Schedules when threads of SIMD instrs. shld run

tracks upto 48 threads

- SIMD has 32768 32-bit registers, divided across SIMD lanes
- Each SIMD thread can use a max of 48 registers

NVIDIA GPU ISA

ensures compatability across generation of GPUs

- Parallel Thread Execution (PTX) instruction are used, hidden to programmer, but speaks to hardware
- PTX mentions operations on a single CUDA thread which can be expanded to many machine instructions
- PTX uses virtual registers
- Optimizer eliminates dead code, folds instrs. together & calculates where branches might diverge

Format of a PTX instruction

opcode.type d, a, b, c; ⇒ 4 operand format

d → destination operand
a, b, c → source operands

Also valid:
opcode;
opcode A;
opcode D, A;
opcode D, A, B;

TYPE	.type SPECIFIER
Untyped bits	.b8, .b16, .b32, .b64
Unsigned integer	.u8, .u16, .u32, .u64
Signed integer	.s8, .s16, .s32, .s64
Floating Point	~~.f8~~, .f16, .f32, .f64

setp.cmp.type
↳ predicate bit p
↳ compare & set predicate
→ 1 ⇒ instruction executed
→ 0 ⇒ instruction not executed

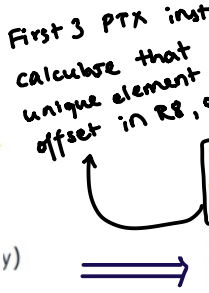
selp.type select with predicate

setp.lt.f32 p, a, b ⇒ p = (a<b);
selp.f32, d, a, b, p ⇒ d = p ? a : b;

Group	Instruction	Example	Meaning	Comments
Arithmetic	arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
	add.type	add.f32 d, a, b	d = a + b;	
	sub.type	sub.f32 d, a, b	d = a – b;	
	mul.type	mul.f32 d, a, b	d = a * b;	
	mad.type	mad.f32 d, a, b, c	d = a * b + c;	multiply-add
	div.type	div.f32 d, a, b	d = a / b;	multiple microinstructions
	rem.type	rem.u32 d, a, b	d = a % b;	integer remainder
	abs.type	abs.f32 d, a	d = |a|;	
	neg.type	neg.f32 d, a	d = 0 - a;	
	min.type	min.f32 d, a, b	d = (a < b)? a:b;	floating selects non-NaN
	max.type	max.f32 d, a, b	d = (a > b)? a:b;	floating selects non-NaN
	setp.cmp.type	setp.lt.f32 p, a, b	p = (a < b);	compare and set predicate
	numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
	mov.type	mov.b32 d, a	d = a;	move
	selp.type	selp.f32 d, a, b, p	d = p? a: b;	select with predicate
	cvt.dtype.atype	cvt.f32.s32 d, a	d = convert(a);	convert atype to dtype
Special Function	special .type = .f32 (some .f64)			
	rcp.type	rcp.f32 d, a	d = 1/a;	reciprocal
	sqrt.type	sqrt.f32 d, a	d = sqrt(a);	square root
	rsqrt.type	rsqrt.f32 d, a	d = 1/sqrt(a);	reciprocal square root
	sin.type	sin.f32 d, a	d = sin(a);	sine
	cos.type	cos.f32 d, a	d = cos(a);	cosine
	lg2.type	lg2.f32 d, a	d = log(a)/log(2)	binary logarithm
	ex2.type	ex2.f32 d, a	d = 2 ** a;	binary exponential
Logical	logic.type = .pred,.b32, .b64			
	and.type	and.b32 d, a, b	d = a & b;	
	or.type	or.b32 d, a, b	d = a | b;	
	xor.type	xor.b32 d, a, b	d = a ^ b;	
	not.type	not.b32 d, a, b	d = ~a;	one's complement
	cnot.type	cnot.b32 d, a, b	d = (a==0)? 1:0;	C logical not
	shl.type	shl.b32 d, a, b	d = a << b;	shift left
	shr.type	shr.s32 d, a, b	d = a >> b;	shift right
Memory Access	memory.space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
	ld.space.type	ld.global.b32 d, [a+off]	d = *(a+off);	load from memory space
	st.space.type	st.shared.b32 [d+off], a	*(d+off) = a;	store to memory space
	tex.nd.dtyp.btype	tex.2d.v4.f32.f32 d, a, b	d = tex2d(a, b);	texture lookup
	atom.spc.op.type	atom.global.add.u32 d,[a], b atom.global.cas.b32 d,[a], b,	atomic { d = *a; *a = cop(*a, b); }	atomic read-modify-write operation
	atom.op = and, or, xor, add, min, max, exch, cas; .spc = .global; .type = .b32			
Control Flow	branch	@p bra target	if (p) goto target;	conditional branch
	call	call (ret), func, (params)	ret = func(params);	call function
	ret	ret	return;	return from function call
	bar.sync	bar.sync d	wait for threads	barrier synchronization
	exit	exit	exit;	terminate thread execution

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
   daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__device__
void daxpy(int n, double a, double *x, double *y)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
   if (i < n) y[i] = a*x[i] + y[i];
}
```

y)



Single iteration

```
shl.u32 R8, blockIdx, 9   ; Thread Block ID * Block size (512 or 2^9)
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
shl.u32 R8, R8, 3         ; byte offset
ld.global.f64 RD0, [X+R8] ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8] ; RD2 = Y[i]
mul.f64 RD0, RD0, RD4     ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64 RD0, RD0, RD2     ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0 ; Y[i] = sum (X[i]*a + Y[i])
```

Predication execution

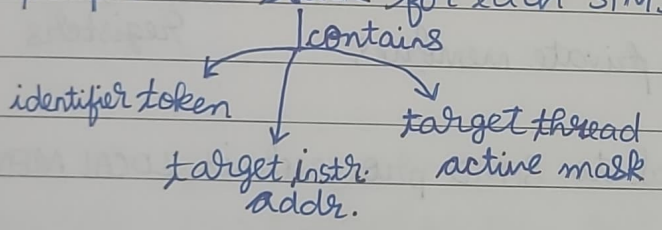
C code : if (i<n)
j = j+1 j

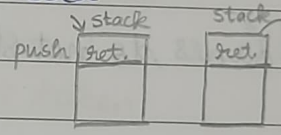
PTX : setp.lt.s32 p,i,n; // p = i<n
@p add.s32 j,j,1 // if i<n, add 1 to j

↳ Guard predicate

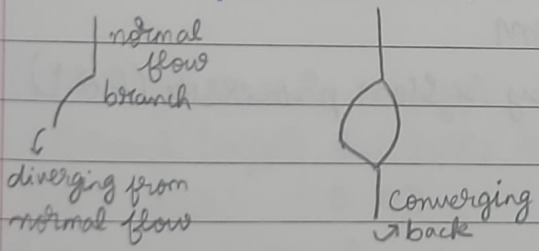
CONDITIONAL BRANCHING IN GPUs

- GPUs handle IF statements with more involvement of hardware
  - ↳ predicate registers (conditional execution)
  - ↳ internal masks
  - ↳ branch synchronisation stack
  - ↳ instruction markers during divergence & convergence
- Control flow includes branch, jump, jump indexed, call, call indexed, return and exit and instructions that manage branch synchronization stack
- GPU provides stack for each SIMD thread

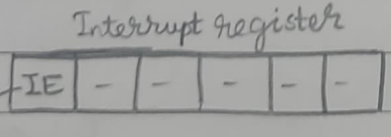




- Per lane predication possible by 1 bit predicate register



↳ set as 1 ⇒ condition satisfied; execute the instr.
↳ set as 0 ⇒ instr. not executed



interrupt enabled
if masked, the values in the other fields is irrelevant

INTERNAL MASK BIT
↳ flips the mask bits (1s to 0s, 0s to 1s)
↳ to take care of instrs. that were not executed (to execute ELSE part)

* Even though the tasks are being delegated to multiple threads, the efficiency might be reduced.

BRANCH DIVERGENCE

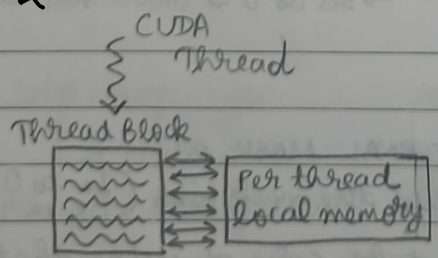
- hardware tracks which microthreads take or don't take branch
- If all go the same way, then keep going in SIMD fashion

CONDITIONAL BRANCHING

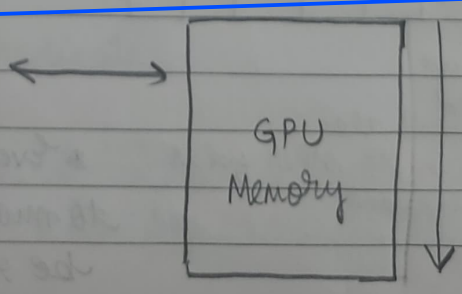
Branches
- unconditional
- uniform
- non-uniform

→ Assume dynamic branching is done, core executes all code path & discards the ones not satisfying condition. So ALU waste time. We fix by predicating GPU instr. instead of GPU branch instrns.

* Each lane in a SIMD processor given with private space in off-chip DRAM memory ⇒ PRIVATE MEMORY
  - used for
    - register spilling
    - stack frame
    - private variable that doesn't fit in registers
* Lanes do not share private memories
  - Unavailable to Host
* Memory that is local to SIMD processor is LOCAL MEMORY
  - & on-chip memory. Shared by SIMD lanes & not process-or
* GPU MEMORY
  - accessed by all thread blocks
  - is off-chip DRAM
  - is accessed by system processor (Host)
  - Host can read & write memory



Smaller streaming Cache
- GPU use S.S Cache & MT SIMD to hide latency to DRAM
- Using that time/latency, chip area used for caches in system processors is spent on computing



VMIPS holds entire vectors in contiguous block of 64 doubles
no multithreading
Single vector distributed across all registers of SIMD Lane

VECTOR ARCHITECTURE	GPU
Vectorized Loop	Grid
Chime (4-16 CC)	Chime (2-4 CC)
Vector instruction	PTX Instruction
Gather/scatter	Global load/store
Mask registers (only software)	Predicate & Internal Mask Registers (Branch synchronization)
Vector Processor	Multithreaded SIMD Processor
Vector Lane (2-8 lanes)	SIMD Lane (8-16 lanes)

Vector latency paid per vector load/store

Hide memory latency using multithreading

Example 1: Determine total branch penalty for a branch target buffer by assuming the penalty cycles for individual mispredictions considering the following details:

INSTRUCTION IN BUFFER	PREDICTION	ACTUAL BRANCH	PENALTY CYCLES
Yes	Taken	Taken	0
Yes	Taken	Not taken	2
No		Taken	2
No		Not taken	0

Make the following assumptions about prediction accuracy & hit rate:
- prediction acc 90% (for instrs in buffer)
- hit rate in buffer 90% (for branches predicted Taken)

Probability (branch in buffer, but not actually taken)
= Percent buffer hit rate × Percent incorrect predictions
= 90% × 10% = 0.09

Probability (branch not in buffer, but actually taken) = 10% = 0.1

Penalty = (0.09 + 0.1) × 2 = 0.38

Example 2

How many bits are in the (0,2) branch predictor with 4K entries? How many entries are in a (2,2) predictor with the same number of bits?

4K entries → 2^{0} (0,2) × $2^{\bullet}$ (2 bits per entry) × 4K = 8 Kbits

(2,2) predictor ⇒ 2^{2} (2,2) × $2^{\bullet}$ × Number of entries = 8 K bits

Number of entries = 1K

15/4/24

LOOP LEVEL PARALLELISM

- usually done at source level ; ILP done after compiling
- involves determining what dependences exist among the operands in a loop across the iterations
- focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iteration ⇒ LOOP-CARRIED DEPENDENCE

```
for ( i = 0 ; i < 100 ; i++ )
    x[i] = x[i] + s ;
```

→ induction variable is the only source of loop-carried dependence; can be easily recognized & elimination

↳ dependence is within a single iteration ; not loop carried

Instead

```
for (i = 0 ; i < 100 ; i++ )
  { x[i] = x[i-1] + s ; }
```

↳ Now loop-carried

```
for (i=0 ; i<100 ; i=i+1)
{   A[i+1] = A[i] + C[i] ;   S1
    B[i+1] = B[i] + A[i+1] ; S2
}
```

} 2 different dependences

```
for(i=0 ; i<100 ; i=i+1)
{   A[i] = A[i] + B[i] ;   S1
    B[i+1] = C[i] + D[i] ; S2
}
```

} dependence is not circular

S1 depends on S2

S2 does not depend on S1

```
A[0] = A[0] + B[0];
for (i=0; i<99; i=i+1){
    B[i+1] = C[i] + D[i];
    A[i+1] = A[i+1] + B[i+1];
}
B[100] = C[99] + D[99];
```

⇒ iterations of the loop may be overlapped, provided statements in each iteration are kept in order

whatever was produced, is being consumed in the same iteration itself; there is no loop-carried dependence

```
for (i=1; i<100; i=i+1)
{
    Y[i] = Y[i-1] + Y[i];
}
```

→ Variable is defined based on value of variable in prev. iteration

⇒ Recurrence relation

~~TLP~~ LLP cannot be exploited

GCD Test : for absence of a dependence

```
for (i=0; i<100; i=i+1){
    X[2*i+3] = X[2*i] * 5.0;
}
```

a = 2, b = 3 (involved in), c = 2, d = 0

∵ it is not associated with loop variable, it is a constant
↳ does not affect the position

GCD (a, c) = 2

d - b = -3

-3 / +3 results in a remainder ⇒ dependence is not possible

* No remainder ⇒ dependence possible

A dependence exists if

(i) there are 2 iteration indices, j and k, that are both within the limits of the ~~for~~ for loop $m \leq j \leq n$; $m \leq k \leq n$

(ii) the loop stores into an array element indexed by $a \times j + b$ & later fetches same array element indexed by $c \times k + d$ such that $a \times j + b = c \times k + d$

Example

```
for (i = 0; i < 100; i = i+1) {
    Y[i] = X[i] / c;      S1
    X[i] = X[i] + c;      S2
    Z[i] = Y[i] + c;      S3
    Y[i] = c - Y[i];      S4
}
```

Find all - true
- output
- anti
dependences

S1-S2 ⇒ WAR Can be executed in parallel (anti dependence) Not true data dependence

S1-S3 ⇒ RAW True data dependence

S1-S4 ⇒ RAW True data dependence

S1-S4 ⇒ WAW Not true data dependence (output dependence)

S2-S3 → Can be executed in parallel

S2-S4 → Can be executed in parallel

S3-S4 ⇒ WAR Not true data dependence

Renamed

```
for (i = 0; i < 100; i = i+1) {
    T[i] = X[i] / c;
    X1[i] = X[i] + c;
    Z[i] = T[i] + c;
    Y[i] = c - T[i]; }
```

Multiprocessors – Computers consisting tightly/loosely coupled processors running multiple threads

– Freq wall, ILP wall, Power density wall, Memory wall

THREAD-LEVEL PARALLELISM

Memory
- Global → Available memory accessed/shared by all processors
- Distributed → Available memory divided and assigned to different processors

	Shared variables	Message passing
Global memory	GMSV	GMMP (Rarely used)
Distributed memory	DMSV	DMMP

Tightly-coupled shared-memory multiprocessors
↳ a set of processors whose coordination & usage are typically controlled by a single OS and that share memory through a shared address space
↓ high degree of coupling

SOFTWARE MODELS
- Parallel processing → The execution of a tightly coupled set of threads collaborating on a single task
- Request-level parallelism

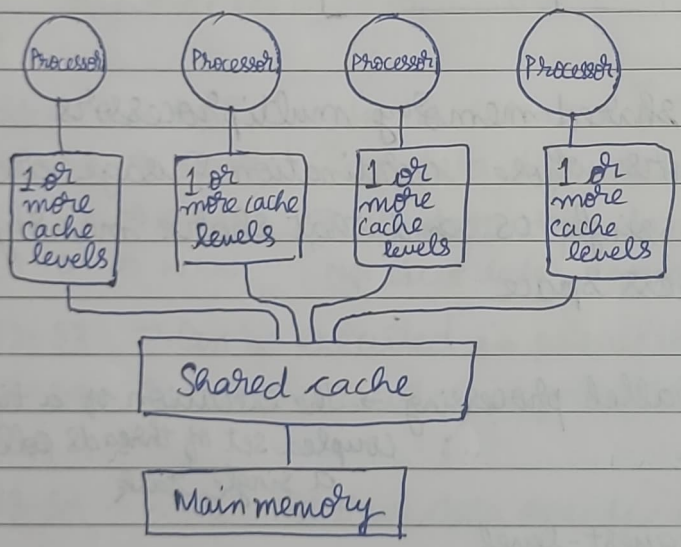
2 Models for Communication
- MESSAGE-PASSING MULTIPROCESSORS Communication occurs by explicitly passing messages among the processors
- SHARED-MEMORY MULTIPROCESSORS Communication occurs through a shared memory space (via loads & stores)

Classes of multiprocessors w.r.t Memory
↳ Centralized Memory Multiprocessor
↳ Physically Distributed-Memory Multiprocessor

16/4/24

SYMMETRIC MULTIPROCESSORS (SMP)

- a centralized memory
- small number of cores (< 32)
- share single memory with uniform memory latency
- uniform memory access for each processor



DISTRIBUTED SHARED MEMORY (DSM)

- Memory is distributed among processors
- Non-uniform memory access/latency (NUMA)

Ex 1 Suppose 80× speedup can be achieved from 100 processors. Find what fraction of original program can be sequential.

AMDAHL'S LAW $\quad Speedup_{overall} = \dfrac{1}{(1 - Fraction_{enhanced}) + \dfrac{Fraction_{enhanced}}{Speedup_{enhanced}}}$

$$80 = \frac{1}{(1 - Fraction_{parallel}) + \dfrac{Fraction_{parallel}}{100}}$$

$$80 \times \left\{(1 - Fraction_{parallel}) + \frac{Fraction_{parallel}}{100}\right\} = 1$$

$$80 - 80\,Fraction_{parallel} + \frac{80}{100} Fraction_{parallel} = 1$$

$$79 = 79.2\,Fraction_{parallel}$$

$$Fraction_{parallel} = 99.75\%$$

Thus to achieve a speedup of 80 with 100 processors, only 0.25% of the original computation can be sequential

Ex 2 Suppose 32-core MP working with 4GHz clock takes 100 ns to access data from remote memory. All local accesses hit memory hierarchy and base CPI is 0.5. What is the performance impact if 0.2% instructions involve remote access?

Clock rate $= \dfrac{1}{4 \times 10^9} = 2.5 \times 10^{-10}\,s = 0.25\,ns$

1 cycle → 0.25 ns

400 cycles → 100 ns

Remote access ⇒ Total CPI = Base CPI + Miss rate × Miss penalty

$= 0.5 + 0.2\% \times 400$

$= 1.3$

$\frac{1.3}{0.5} = 2.6 \Rightarrow$ performance impact

APPLICATION PARALLELISM → Primarily via new algorithms that have better parallel performance

LONG REMOTE LATENCY IMPACT → both by architect and by the programmer

Multiprocessors usually cache both private data (used by a single processor) & shared data (used by multiple processors)

- Caching shared data can reduce latency and required memory bandwidth (due to local access)
- But caching shared data introduces CACHE COHERENCE PROBLEM
  ↳ not maintaining the same data across the memory hierarchies and between the processors

GCD

```
for (int i=0; i<100; i++)
{
S1  a[2*i] = b[i];
S2  c[i] = a[4*i+1]; }
```

Find if loop dependence exists between S1 and S2

$a[2*i] = a[4*i+1]$

$a*i+j = b*i+k$

$a=2 \quad b=0 \quad c=4 \quad d=1$

$GCD(a,c) = (2,4) = 2 \qquad d-b = 1-0 = 1$

$\frac{GCD(a,c)}{d-b} = \frac{2}{1} = 2 \quad \frac{1}{2} \Rightarrow$ no dependence possible

```
for(int i=0; i<n; i++)
{
 S1  a[x*i+k] = ... ;
 S2  ...      = a[y*i+m]; }
```

a = x b = K c = y d = m

GCD(x, y) m − K

17/4/24

COHERENCE defines values returned by a read
→ Write to the same location as seen by the other processors

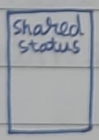
Defining Coherent Memory System
1) Preserve program order
2) Coherent view of memory
3) Write serialization

Classes of Cache Coherence Protocols
- HW → use HW to track shared data status
- SW

Basic Schemes for Enforcing Coherence (to track the sharing status)

- DIRECTORY-BASED · Sharing status of a block of physical memory is kept in just one location, the directory
  · centralized
- SNOOPING · Every cache with a copy of data also has a copy of sharing status of block, but no centralized state is kept
  · Distributed

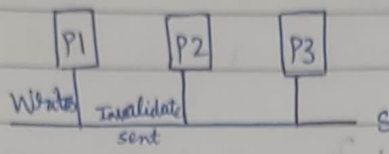
Directory



Snooping Coherence Protocols

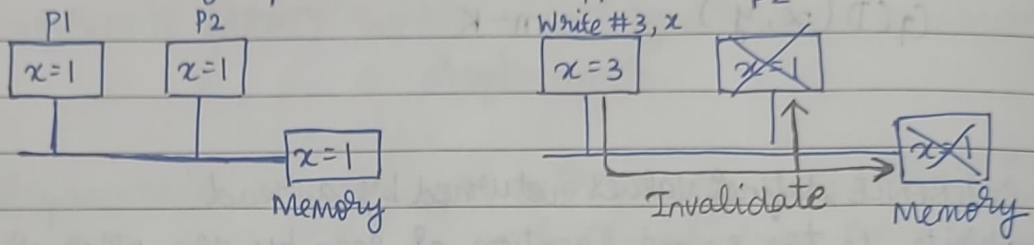
Write Invalidate
Write to shared data → Invalidate is sent to all caches which snoop → Invalidate any copies
Read miss → Write Through / Write Back

Write Broadcast
Write to shared data → Broadcast on bus → Processors Snoop → Update copies



→ Only one processor writes at a time

WRITE SERIALIZATION Bus serializes requests
→ Bus is single point of arbitration



WRITE-THROUGH Get up-to-date copy from lower level cache or memory (simpler, but enough memory BW is necessary)

WRITE-BACK • Modified value maybe in cache,
When another processor requests cache supplies updated value

18/4/24

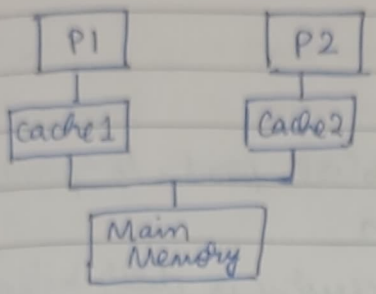
WB Snoopy Protocol

- Invalidation protocol, write-back cache
- → Snoops every address on bus
- → If it has a dirty copy of requested block, provides that block in response to the read request and aborts the memory access

Each memory block is in one of 3 states

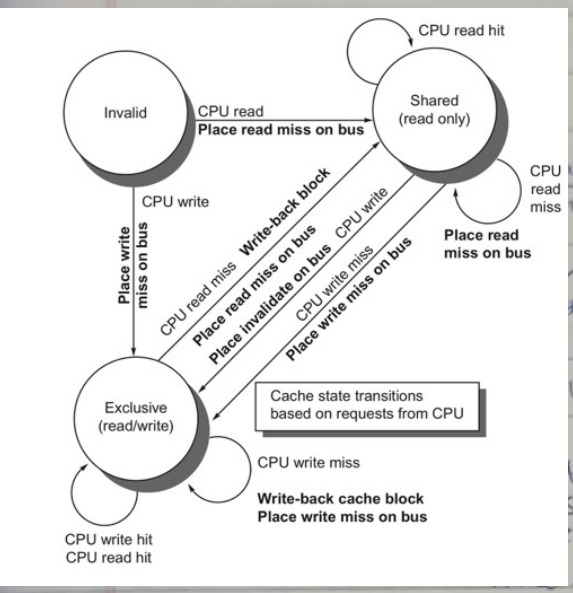
MSI Modified Shared Invalidate
MESI Modified Exclusive Shared Invalidate (Exclusive)
MEOSI Modified Exclusive Owned Shared Invalidate

If processor reads & then writes same bl



When in Shared State,

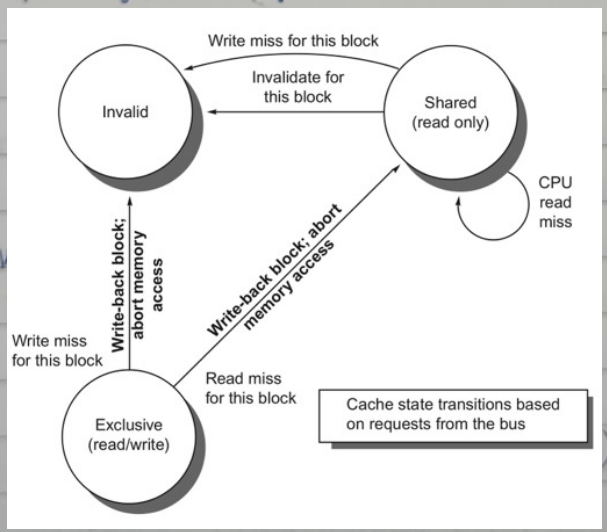
- Write operation in Modified State
- When there is a READ MISS, processor having the correct value of X is in Modified/ Exclusive state moves to Shared state and puts the data onto the shared bus



CPU Read hit
Shared (read/ only)
CPU Read miss
Place Read miss on bus
Write
Write Miss on Bus
Write
back cache block
write miss on bus

* Read hit not possible in Invalid state
* In shared state, value being read is up-to-date value

* Write hit possible only when in Exclusive state, after placing/putting Invalidate on the shared bus

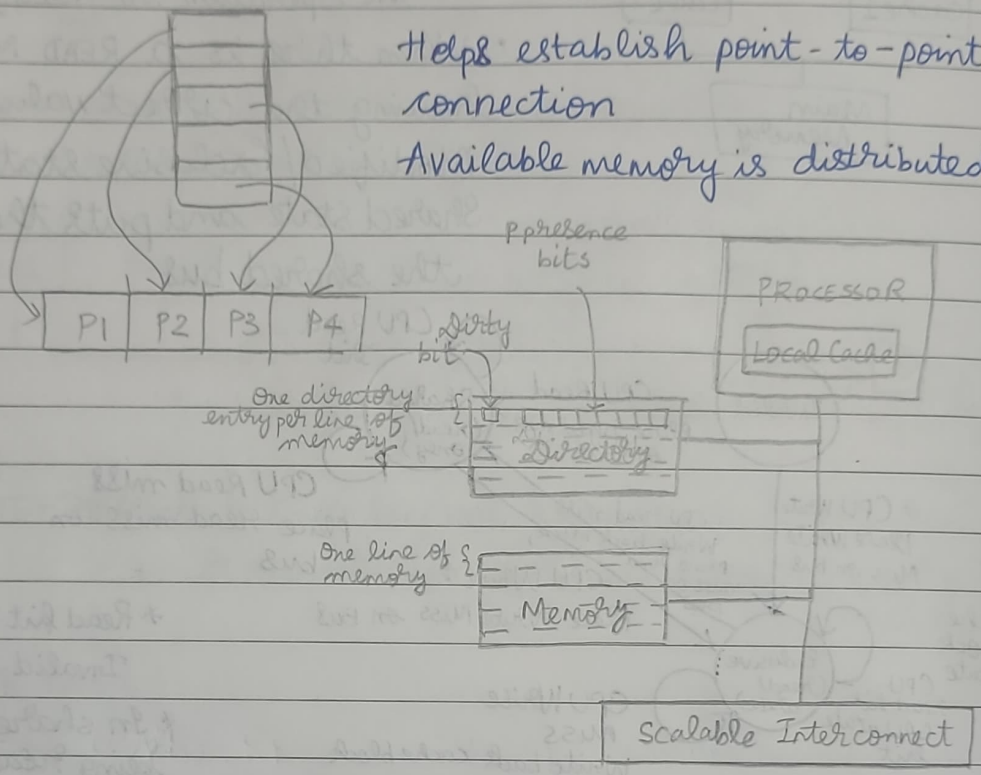


- Snoopy schemes broadcast requests over memory bus
- Difficult to scale to large numbers of processors
- Requires additional bandwidth to cache tags for Snoop requests

Shared → Shared
Exclusive → Cached
Invalid → Uncached

22/4/24

DIRECTORY-BASED COHERENCE



Helps establish point-to-point connection
Available memory is distributed

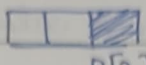
DIRTY BIT : Indicates line is dirty in one of the processors' caches
LINE : A region of memory that would be cached as a single block

NODES
- LOCAL NODE — Node where a request originates
- HOME NODE — Node where memory location of an address resides
- REMOTE NODE — Node that has a copy of a cache block, whether exclusive or shared

Scenario 1
Read miss to clean line {D=0}
- Processor 0 → Local node
- Processor 1 → Home node — Clean block {D = 0}

⇒ Receives Read miss msg {Sent from local to home node}
⇒ Responds with line of requested data from memory; presence[0] set to TRUE
- Copy of requested data stored in Local Cache of Processor 0

Scenario 2 (Read miss to dirty line)

Processor 0 → Local node
Processor 1 → Home node
Request: Read miss msg to Processor 0
Dirty bit set  Presence[2] set ⇒ up-to-date info in processor 2

owner ID obtained
Requested data present in local cache of Processor 2
Data put onto line, updated in Processor 1 memory and send to Processor 0, where it is updated in its local cache

Scenario 3 (Write miss to clean line)

- Processor 0 wants to write in a block of data originating in Processor 1 and having a copy in Processor 2
- Shared ids {presence[1] and presence[2]} + data sent to Processor 0
- Processor 0 sends Invalidate msg to the shared ids

NOT THERE IN 2023 batch Syllabus

SYNCHRONIZATION

SYNCHRONIZATION : necessary for coordination of complex activities and multi-threading

ATOMIC ACTIONS : actions that cannot be interrupted

LOCKS : mechanisms to protect a critical section, code that can be executed by only one thread at a time

Hardware support

Thread coordination
⇒ multiple threads of a thread group need to act in concert

Mutual exclusion
⇒ only 1 thread at a time should be allowed to perform an action

LOCK
0 → available (unlocked)
1 → locked

Atomicity requires hardware support

→ TEST-AND-SET Instruction which writes to a memory location and returns the old content of that memory cell as non-interruptible

LL ⇒ Load Lock Read Lock value and set to 1

→ COMPARE-AND-SWAP Instruction which compares the contents of a memory location to a given value and, only if the 2 values are the same, modifies the contents of that memory location to a given new value

load linked / load locked } store conditional
Store ~~cond~~

ATOMIC EXCHANGE

```
try: MOV R3, R4      mov exchange value
     LL R2, 0(R1)    load linked
     SC R3, 0(R1)    store conditional
     BEQZ R3, try    branch store fails
     MOV R4, R2      put load value in R4
```

} Contents of R4 and memory location specified by R1 have been atomically exchanged

ATOMIC FETCH-AND-INCREMENT

```
try: LL R2, 0(R1)
     DADDUI R3, R2, #1
     SC R3, 0(R1)
     BEQZ R3, try
```

Spin lock a lock the system keeps trying to acquire
The system supports cache coherence
- When a thread tries to acquire a lock, it will use the local copy rather than requiring a global memory access
- Locality of lock access → a thread that used a lock is likely to need it again

Spin lock

No coherence

```
        DADDUI  R2, R0, #1
lockit  EXCH    R2, 0(R1)     ; atomic exchange
        BNEZ    R2, lockit    already locked?
```

Coherence

```
lockit: LD      R2, 0(R1)
        BNEZ    R2, lockit
        DADDUI  R2, R0, #1
        EXCH    R2, 0(R1)
        BNEZ    R2, lockit
```

Sequential Consistency

The result of any execution will be the same as if the memory accesses by each processor ~~be~~ were kept in order and the accesses ~~be~~ were interleaved arbitrarily

Example Suppose we have a processor where a write miss takes 50 cycles to establish ownership, 10 cycles to issue each invalidate after ownership is established, and 80 cycles for an invalidate to complete & be acknowledged once it is issued. Assuming that 4 other processors share a cache block, how long does a write miss stall the writing processor if the processor is sequentially consistent?
Invalidates must be explicitly acknowledged.

$$50 + \overbrace{(10 + 10 + 10 + 10)}^{\text{4 processors}} + 80 = 170 \text{ cycles} \Rightarrow \text{Sequentially Consistent}$$

$50 + 10 + 80 = 140$ cycles → Relaxation in sequential consistent approaches

- hardware
- software

Relaxed consistency models

RULES $X \rightarrow Y$ Operation X must complete before operation Y is done

$R \rightarrow W$
$R \rightarrow R$
$W \rightarrow R$
$W \rightarrow W$
} Sequential consistency requires all 4

RELAXING

1. Relax $W \rightarrow R$ "Total store ordering"
2. Relax $W \rightarrow W$ "Partial store order"
3. Relax $R \rightarrow W$ and $R \rightarrow R$
   "Weak ordering" and "release consistency"